

Classificação Supervisionada de Variedades de Feijão

— Dry Bean Dataset (UCI) —

Integrantes: Luciano [SOBRENOME] e [INTEGRANTE 2 — remover se individual]

Disciplina: Aprendizado de Máquina

Data: 11 de junho de 2026

Repositório GitHub: [https://github.com/\[SEU-USUARIO\]/trabalho-dry-bean](https://github.com/[SEU-USUARIO]/trabalho-dry-bean)

1. Apresentação do dataset

O **Dry Bean Dataset** está disponível no repositório UCI Machine Learning Repository em <https://archive.ics.uci.edu/dataset/602/dry+bean+dataset>. O dataset foi construído por Koklu e Ozkan (2020) a partir de imagens de alta resolução de 13.611 grãos de feijão seco de sete variedades registradas na Turquia. Por meio de técnicas de visão computacional, cada grão foi segmentado e dele foram extraídos 16 atributos numéricos: 12 dimensões (área, perímetro, comprimentos dos eixos, excentricidade, etc.) e 4 fatores de forma.

O propósito do dataset é a **classificação automática da variedade do feijão** a partir das características geométricas do grão, tarefa relevante para a certificação de sementes e o controle de qualidade na indústria agrícola, substituindo a inspeção visual manual. O **atributo classe / target é o atributo nominal Class**, que assume sete valores: BARBUNYA, BOMBAY, CALI, DERMASON, HOROZ, SEKER e SIRA. Todos os demais atributos (numéricos) são utilizados como preditores.

2. Caracterização dos dados

O arquivo original (Dry_Bean_Dataset.xlsx) utilizado neste trabalho possui **13.611 instâncias** e **18 atributos**: um identificador sequencial (*Bean ID*, presente no espelho do arquivo utilizado), 16 atributos preditores numéricos e o atributo classe (*Class*, nominal). **Nenhum atributo possui valores faltantes** (0 ausências em todas as colunas) e não há instâncias duplicadas. A Tabela 1 resume o tipo e o intervalo de valores de cada atributo.

Atributo	Tipo	Intervalo / Conjunto de valores	Faltantes
Bean ID	Num. (inteiro)	[1 – 13611]	0
Area	Num. (inteiro)	[20420 – 254616]	0
Perimeter	Num. (real)	[524.7360 – 1985.3700]	0
MajorAxisLength	Num. (real)	[183.6012 – 738.8602]	0
MinorAxisLength	Num. (real)	[122.5127 – 460.1985]	0
AspectRatio	Num. (real)	[1.0249 – 2.4303]	0
Eccentricity	Num. (real)	[0.2190 – 0.9114]	0
ConvexArea	Num. (inteiro)	[20684 – 263261]	0
EquivDiameter	Num. (real)	[161.2438 – 569.3744]	0
Extent	Num. (real)	[0.5553 – 0.8662]	0
Solidity	Num. (real)	[0.9192 – 0.9947]	0
roundness	Num. (real)	[0.4896 – 0.9907]	0
Compactness	Num. (real)	[0.6406 – 0.9873]	0
ShapeFactor1	Num. (real)	[0.0028 – 0.0105]	0

Atributo	Tipo	Intervalo / Conjunto de valores	Faltantes
ShapeFactor2	Num. (real)	[0.0006 – 0.0037]	0
ShapeFactor3	Num. (real)	[0.4103 – 0.9748]	0
ShapeFactor4	Num. (real)	[0.9477 – 0.9997]	0
Class	Nominal	{BARBUNYA, BOMBAY, CALI, DERMASON, HOROZ, SEKER, SIRA}	0

Tabela 1 — Caracterização dos 18 atributos do dataset.

Como os 16 preditores são todos numéricos e contínuos, o conceito de desbalanceamento aplica-se ao único atributo nominal do dataset: a própria classe. O **atributo mais desbalanceado é Class** — a variedade majoritária DERMASON possui 3.546 instâncias (26,05%), enquanto a minoritária BOMBAY possui apenas 522 (3,84%), uma razão de desbalanceamento de **6,79:1**, como mostra a Figura 1.

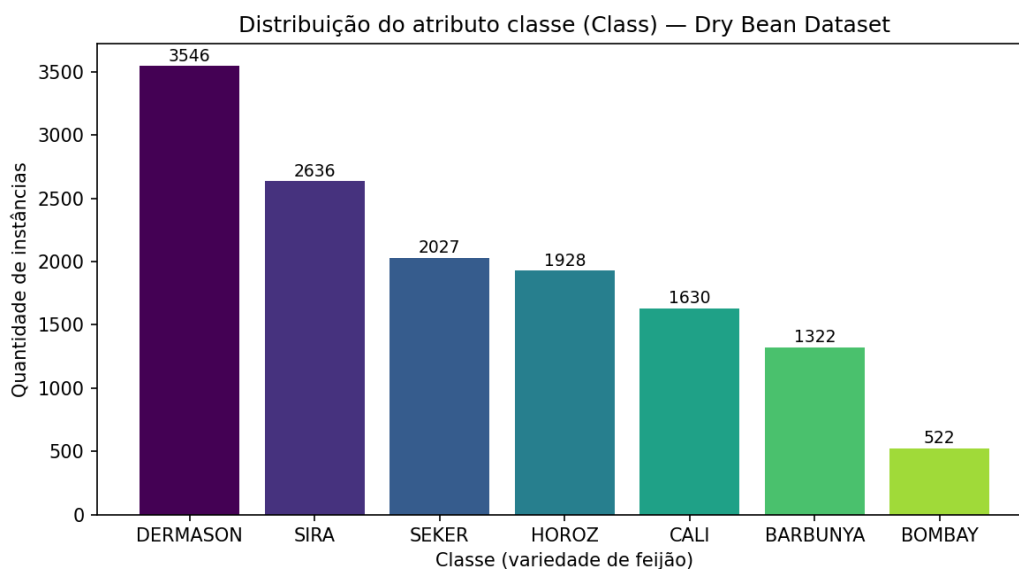


Figura 1 — Distribuição das 7 classes (desbalanceamento de 6,79:1).

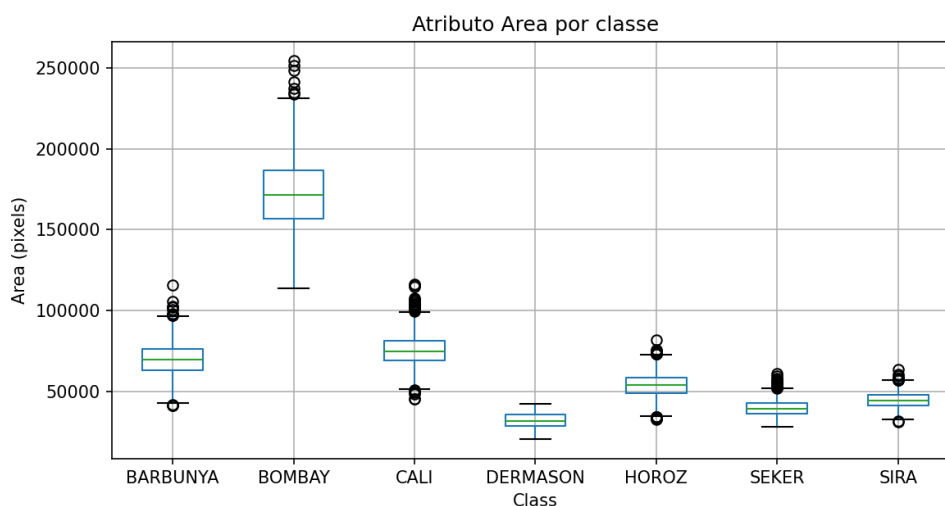


Figura 2 — Atributo Area por classe: BOMBAY é claramente separável; as demais variedades se sobrepõem.

3. Procedimentos de pré-processamento

O atributo classe já é nominal, portanto não foi necessária a discretização do target exigida por alguns algoritmos. Foram aplicadas duas variações de pré-processamento, ambas geradas pelo script `02_experimentos.py` e publicadas no repositório:

P1 — Limpeza mínima (dados/dry_bean_preproc1.csv): remoção do atributo *Bean ID*, que é um identificador sequencial sem valor preditivo (mantê-lo induziria os modelos a memorizar a ordem dos registros, pois o arquivo é ordenado por classe). Os 16 atributos numéricos foram mantidos em suas escalas originais.

P2 — P1 + padronização (dados/dry_bean_preproc2.csv): aplicação de *StandardScaler* (média 0 e desvio-padrão 1 em cada atributo). A motivação é a grande diferença de escala entre atributos — *Area* varia de 20.420 a 254.616 pixels, enquanto *ShapeFactor2* varia de 0,0006 a 0,0037. Algoritmos baseados em distância, como o KNN, são fortemente afetados por essa disparidade, pois os atributos de maior magnitude dominam o cálculo da distância euclidiana.

4. Resultados experimentais

Os três algoritmos — Árvore de Decisão (CART), Random Forest (100 árvores) e KNN (k=5), implementações da biblioteca scikit-learn 1.7 — foram avaliados por **validação cruzada estratificada de 10 folds** (seed=42). As tabelas apresentam a média das métricas nos 10 folds; precisão, recall e F-Measure usam média macro (média simples entre as 7 classes, adequada a dados desbalanceados). O tempo refere-se ao treino médio por fold (12.250 instâncias).

Algoritmo	Acurácia	Precisão	Recall	F-Measure	Tempo de treino (s)
Árvore de Decisão	89.58%	0.911	0.910	0.910	0.1960
KNN (k=5)	73.43%	0.743	0.730	0.734	0.0051
Random Forest	92.59%	0.939	0.935	0.937	3.2915

Tabela 2 — Resultados com pré-processamento P1 (sem normalização).

Algoritmo	Acurácia	Precisão	Recall	F-Measure	Tempo de treino (s)
Árvore de Decisão	89.58%	0.911	0.910	0.910	0.2016
KNN (k=5)	92.33%	0.937	0.934	0.936	0.0051
Random Forest	92.60%	0.939	0.935	0.937	3.3216

Tabela 3 — Resultados com pré-processamento P2 (com padronização).

A comparação entre as tabelas evidencia o papel do pré-processamento: a padronização (P2) não altera os resultados das árvores (Árvore de Decisão e Random Forest são invariantes a transformações monotônicas de escala), mas eleva a acurácia do KNN de **73,43% para 92,33%** (+18,9 pontos percentuais), confirmando a sensibilidade de algoritmos baseados em distância à escala dos atributos. Em tempo de treino, o KNN é o mais rápido (~0,005 s, treino "preguiçoso" que apenas armazena os dados), seguido da Árvore de Decisão (~0,20 s); o Random Forest é o mais custoso (~3,3 s, 100 árvores), cerca de 650× mais lento que o KNN.

5. Análise do melhor e do pior resultado

Melhor algoritmo — Random Forest (P2): acurácia média de **92.60%**, precisão de **0.939**, recall de **0.935** e F-Measure de **0.937**, com tempo médio de treino de 3.32 s. O ensemble de 100 árvores reduz a variância da árvore individual e lida bem com a sobreposição entre classes. O KNN padronizado (92,33%) fica praticamente empatado, com fração do custo de treino.

Pior resultado — KNN sem normalização (P1): acurácia de 73,43% e F-Measure de 0,734, consequência direta da escala desigual dos atributos. Considerando apenas o cenário com padronização (P2), o pior algoritmo é a **Árvore de Decisão** (acurácia de 89,58%, F-Measure de 0,910), que, por ser um modelo único, apresenta maior variância que o ensemble.

A análise por valor do atributo classe (holdout estratificado 75/25, P2) é apresentada na Tabela 4. Em **todos os algoritmos, a classe BOMBAY obteve o melhor resultado, com precisão, recall e F1 perfeitos (1,000)** — seus grãos são muito maiores que os demais (Figura 2), tornando-a linearmente separável. O **pior desempenho ocorre sistematicamente na classe SIRA** (F1 de 0,860 no Random Forest e 0,813 na Árvore de Decisão), confundida principalmente com DERMASON — variedades de geometria semelhante —, como mostra a matriz de confusão (Figura 3). BARBUNYA e DERMASON também ficam abaixo da média do modelo.

Classe	Prec. (RF)	Recall (RF)	F1 (RF)	Prec. (AD)	Recall (AD)	F1 (AD)	n teste
BARBUNYA	0.934	0.894	0.913	0.901	0.882	0.891	330
BOMBAY	1.000	1.000	1.000	1.000	1.000	1.000	130
CALI	0.936	0.931	0.934	0.914	0.912	0.913	408
DERMASON	0.908	0.915	0.912	0.881	0.888	0.885	887
HOROS	0.960	0.952	0.956	0.941	0.927	0.934	482
SEKER	0.931	0.953	0.942	0.917	0.939	0.928	507
SIRA	0.859	0.860	0.860	0.815	0.810	0.813	659

Tabela 4 — Desempenho por classe: melhor algoritmo (RF — Random Forest) e pior em P2 (AD — Árvore de Decisão).

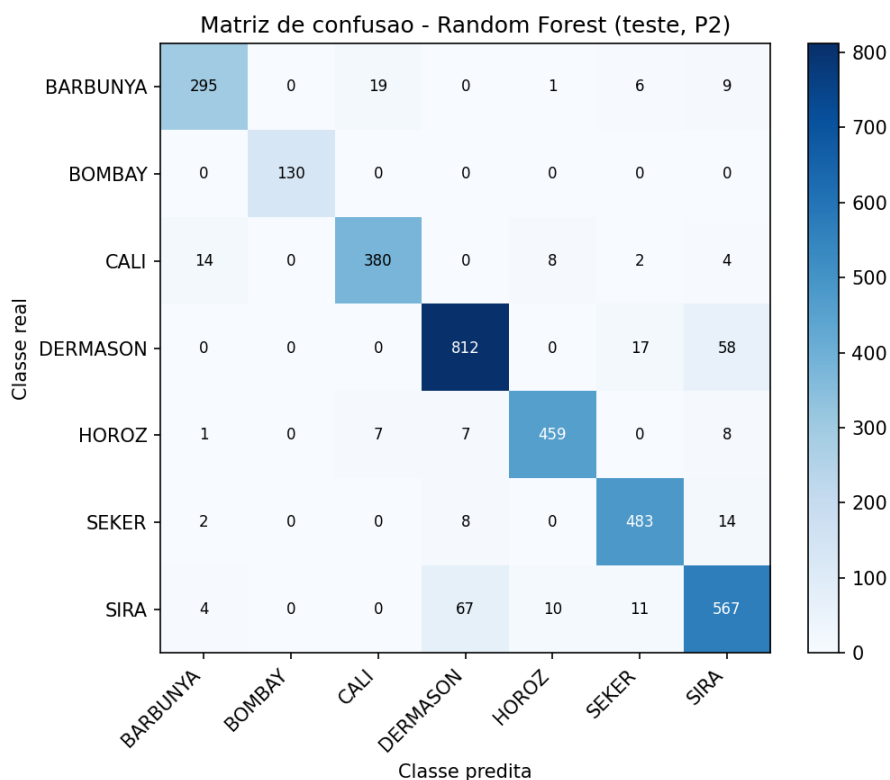


Figura 3 — Matriz de confusão do Random Forest (conjunto de teste, P2): a principal confusão é SIRA × DERMASON.

6. Conclusões

O artigo de origem do dataset (Koklu & Ozkan, 2020), também com validação cruzada de 10 folds, reporta acurácias de 93,13% (SVM), 92,52% (Árvore de Decisão), 91,73% (MLP) e 87,92% (kNN). O nosso melhor resultado — **Random Forest com 92,60%** — é, portanto, compatível com a faixa reportada no repositório de origem, ficando apenas 0,5 ponto percentual abaixo do melhor modelo dos autores (SVM, não incluído em nosso comparativo) sem qualquer ajuste de hiperparâmetros. Nosso KNN com padronização (92,33%) supera o kNN reportado pelos autores (87,92%), o que reforça a importância do pré-processamento P2.

A análise por classe também converge com o estudo original: lá, o SVM acertou 100% de BOMBAY e teve em SIRA seu pior desempenho (86,84%); aqui, BOMBAY foi perfeita ($F1 = 1,000$) e SIRA foi a classe mais difícil ($F1 = 0,860$ no RF). Conclui-se que (i) o desbalanceamento do dataset não impediu bom desempenho macro, (ii) a escolha do pré-processamento é decisiva para algoritmos baseados em distância e (iii) a dificuldade do problema concentra-se na fronteira SIRA × DERMASON, candidata natural a trabalhos futuros (e.g., ajuste de hiperparâmetros ou atributos adicionais de textura).

7. Arquivos publicados no GitHub

O repositório [https://github.com/\[SEU-USUARIO\]/trabalho-dry-bean](https://github.com/[SEU-USUARIO]/trabalho-dry-bean) contém: **dados/** (arquivo original xlsx/csv e os dois pré-processados), **scripts/** (01_exploracao.py, 02_experimentos.py e 03_relatorio.py), **modelos/** (três modelos finais e o scaler, formato joblib), **figuras/** (gráficos gerados) e **logs/** (logs de execução, tabela de caracterização, resultados da validação cruzada e relatórios por classe), além deste relatório em PDF.

Referências

KOKLU, M.; OZKAN, I. A. Multiclass classification of dry beans using computer vision and machine learning techniques. *Computers and Electronics in Agriculture*, v. 174, 105507, 2020.
DRY BEAN DATASET. UCI Machine Learning Repository, 2020. Disponível em: <https://archive.ics.uci.edu/dataset/602/dry+bean+dataset>.
PEDREGOSA, F. et al. Scikit-learn: Machine Learning in Python. *JMLR*, v. 12, p. 2825-2830, 2011.